

NotifyAfrica Console - Spécifications fonctionnelles détaillées

NotifyAfrica

22 septembre 2026

1. Rôle

NotifyAfrica Console est l'espace opérationnel des clients. Elle doit permettre à une organisation de découvrir les services activés, recharger son compte, intégrer les APIs, envoyer des communications, consulter les résultats, demander un devis et administrer son équipe.

2. Stack et domaine

- Next.js / TypeScript ;
- runtime Node ;
- Core API comme source métier ;
- Design System Claude ;
- domaine distinct, par exemple `app.notifyafrica.com` ;
- aucune logique de prix finale dans le frontend.

3. Authentification

Écrans :

- register ;
- login ;
- verification ;
- forgot password ;
- reset ;
- MFA ;
- invitation ;
- session expired.

4. Inscription et événement Admin

À la création d'un utilisateur :

1. Console valide les données ;
2. Core crée le compte ;
3. événement `USER_REGISTERED` ;
4. Notification Engine informe l'Admin selon règles ;
5. vérification ;
6. onboarding.

5. Onboarding

Étapes dynamiques :

1. profil ;
2. organisation ;
3. informations entreprise ;
4. pays/devise ;
5. projet ;
6. choix de services ;
7. API key ou première action ;
8. recharge facultative/obligatoire selon service ;
9. fin.

L'Admin doit pouvoir activer/désactiver certaines étapes.

6. Organisation et projets

Un user peut appartenir à plusieurs organisations.

Une organisation possède plusieurs projets.

Switchers :

- Organization ;
- Project ;
- Environment.

7. Dashboard

Widgets issus de l'API :

- balance ;
- reserved balance ;
- spend ;
- usage ;

- volumes ;
- taux succès ;
- erreurs ;
- devis ;
- incidents ;
- activités récentes.

Les widgets dépendent des produits activés.

8. Wallet

Affichage :

- available ;
- reserved ;
- pending ;
- currency ;
- last transactions.

Actions :

- add funds ;
- transactions ;
- usage ;
- statement.

9. Recharge

Les moyens disponibles sont retournés par l'API selon pays et organisation.

Flux :

1. montant ;
2. méthode ;
3. éventuels frais ;
4. confirmation ;
5. paiement ;
6. statut ;
7. crédit wallet.

10. SMS - envoi simple

Champs :

- sender ;
- destination ;
- message ;

- schedule.

Avant confirmation :

- segmentation ;
- estimation backend ;
- unit price ;
- remise ;
- taxes ;
- total ;
- balance after.

11. SMS - Bulk

Sources audience :

- saisie ;
- CSV ;
- contacts ;
- segments.

Validation :

- doublons ;
- invalides ;
- blacklist ;
- pays ;
- volume ;
- estimation.

12. Campaigns

Flow :

Draft -> Channel -> Audience -> Content -> Estimate -> Schedule -> Review
-> Reserve Funds -> Run -> Report.

Statuts :

- draft ;
- scheduled ;
- queued ;
- running ;
- paused ;
- completed ;
- partial ;
- failed ;
- cancelled.

13. SMS History

Filtres :

- date ;
- status ;
- sender ;
- destination ;
- campaign ;
- project.

Détail :

- request id ;
- provider status normalisé ;
- timestamps ;
- cost ;
- delivery ;
- error.

14. Sender IDs

Flux :

Draft -> Submit -> Review -> Provider Review -> Approved/Rejected.

Les exigences du formulaire proviennent de la configuration pays/provider.

15. OTP

Configuration :

- app ;
- length ;
- expiry ;
- attempts ;
- resend ;
- channel ;
- fallback ;
- template.

Les limites viennent du backend.

16. WhatsApp

Modules :

- overview ;

- numbers ;
- templates ;
- send ;
- logs ;
- analytics ;
- pricing.

Avant envoi : estimation dynamique selon catégorie, pays, volume, client.

17. Email

Modules :

- domains ;
- identities ;
- templates ;
- send ;
- logs ;
- suppressions ;
- analytics.

18. Pricing & Quotes

La Console affiche :

- pricing public ;
- pricing contractuel ;
- remises actives ;
- simulateur ;
- demandes de devis ;
- devis reçus.

19. Quote Builder

Les champs sont dynamiques par produit.

Exemple : SMS peut demander pays + volume ; Payment API peut demander volume transactionnel + montant moyen + opérateur.

Statuts synchronisés avec Admin.

20. Devis reçu

Détail :

- référence ;

- service ;
- conditions ;
- volume ;
- prix ;
- taxes ;
- durée ;
- validité ;
- PDF ;
- accepter/refuser.

Une acceptation peut activer automatiquement une règle tarifaire spécifique après validation.

21. Developers

Modules :

- API Keys ;
- Webhooks ;
- API Logs ;
- Usage ;
- SDKs ;
- Docs ;
- Sandbox.

22. API Keys

Champs :

- name ;
- environment ;
- scopes ;
- expiry ;
- IP restrictions.

Secret affiché une seule fois.

23. Webhooks

Gestion :

- URL ;
- secret ;
- events ;
- status ;
- attempts ;

- replay selon permission.

24. Logs API

Filtres :

- endpoint ;
- status ;
- date ;
- latency ;
- request id.

Masquer les données sensibles.

25. Future Payment APIs

La Console doit pouvoir afficher de nouveaux produits API sans refonte totale.

Modules possibles :

- Collection ;
- Payout ;
- Transactions ;
- Refund ;
- Reversal ;
- Balance.

Les menus doivent être pilotés par le catalogue et les feature flags.

26. Transactions Payment futures

Détail normalisé :

- transaction ID ;
- client reference ;
- amount ;
- currency ;
- operator ;
- status ;
- fees ;
- timestamps ;
- callback status.

27. Analytics

Filtres dynamiques :

- période ;
- projet ;
- produit ;
- pays ;
- opérateur ;
- statut.

KPIs selon produit.

28. Billing

Modules :

- Overview ;
- Wallet ;
- Usage ;
- Transactions ;
- Invoices ;
- Payment Methods ;
- Quotes.

29. Team

Fonctions :

- members ;
- invitations ;
- roles ;
- permissions visibles ;
- remove ;
- ownership transfer.

30. Settings

Organisation :

- profile ;
- legal ;
- country ;
- currency ;
- timezone ;
- security.

Project :

- services ;
- environment ;

- webhooks ;
- limits.

User :

- profile ;
- password ;
- MFA ;
- sessions ;
- notifications.

31. Support

- new ticket ;
- history ;
- attachments ;
- replies ;
- incident notices ;
- help center.

32. Notifications client

Événements :

- low balance ;
- quote available ;
- campaign finished ;
- key expiring ;
- incident ;
- payment completed/failed.

33. Fallback si Backoffice vide

La Console doit pouvoir démarrer avec le seed issu du design Claude.

Exemples :

- navigation initiale ;
- catalog initial ;
- pricing initial ;
- formulaires initiaux ;
- textes d'onboarding.

Le frontend lit tout via un `Config Client` partagé ; il ne lit pas directement des constantes locales.

34. Gestion des erreurs

États minimum :

- insufficient balance ;
- pricing unavailable ;
- quote required ;
- provider unavailable ;
- validation error ;
- unauthorized ;
- permission denied ;
- rate limited ;
- timeout ;
- maintenance.

35. Responsive

Desktop principal. Mobile doit au minimum permettre :

- dashboard ;
- balance ;
- monitoring ;
- logs ;
- support ;
- transactions ;
- quick send si approprié.

36. Critères d'acceptation

- inscription déclenche notification Admin ;
- organisation/projet fonctionnels ;
- pricing provient du backend ;
- wallet contrôlé avant opération ;
- menus produits sont dynamiques ;
- devis synchronisés ;
- seed fonctionne sans données Admin ;
- API keys sécurisées ;
- futur produit Payment peut être activé par configuration ;
- déploiement domaine distinct.